

NoPS

The deterministic decision-intelligence platform. Upload a spreadsheet, get board-ready dashboards, forecasts, alerts and answers — every number reproducible and auditable.

Formerly “DecisionIQ” · A single, honest source of truth for a project that grew fast

What’s in this handbook

Read the first three sections for the plain-English version of “what the hell is actually happening.” Everything after is progressively more technical — dip in as deep as you need.

1	The 60-second version	10	The technology stack
2	What NoPS is (plain English)	11	The database (what’s stored)
3	The one big idea: “deterministic”	12	Security & privacy
4	What it actually does — feature tour	13	Testing & quality
5	Who it’s for & how it makes money	14	The big future bet — generative design
6	The story so far — full timeline	15	Roadmap — what’s next
7	Current status — what’s real today	16	Every file, explained (useful vs not)
8	How it’s built — architecture	17	How to run it yourself
9	The engine (the crown jewels)	18	Honest assessment & risks
		19	Glossary for non-technical readers

84

COMMITTS

22

MERGED PRS

~12k

LINES OF CODE

16

TEST SUITES

HOW TO READ THE STATUS LABELS

BUILT

 shipped and working

PARTIAL

 exists, needs extending

GAP

 not started

PLANNED

 designed, not built

The 60-second version

If you read nothing else, read this.

IN ONE SENTENCE

NoPS is a web app where a business person uploads a spreadsheet of their sales/operations data and instantly gets a professional analytics dashboard, forecasts, automatic alerts, plain-English answers to questions, and a downloadable PDF report — without hiring a data analyst.

The five things that matter

1. **It's a real, working product**, not a prototype. You can run it today with one command and log in with a demo account. It has real users' concerns baked in: accounts, teams, billing tiers, security.
2. **Its whole selling point is trust**. Every number it shows is calculated by plain, readable code that gives the same answer every time — never guessed by an AI. That's the meaning of the word “deterministic” you'll see everywhere. It's aimed at people who can't afford a chatbot to make up a figure (finance, regulated industries).
3. **AI is used carefully and optionally**. The only place AI appears is to *understand your typed question* (“which products are declining?”) and translate it into an instruction. The actual math is still done by the trustworthy code. And your data rows are *never* sent to any AI — only the question text and column names.
4. **It has been renamed once**. It started life as “**DecisionIQ**” and was renamed to “**NoPS**” in August 2026. Same product, newer name. You'll still find “DecisionIQ” in older commit history.
5. **There's a much bigger vision written down but not yet built**: a “generative design system” that would auto-design a *uniquely styled* dashboard for each business (a restaurant vs. a fintech get different looks), while keeping the number-crunching identical and trustworthy. Today the dashboard adapts its *content* per industry, but not yet its *visual design*.

Where it is right now

A feature-complete MVP (minimum viable product) with an unusually strong engineering foundation for its age: multi-tenant accounts, role-based permissions, a tested deterministic analytics engine, forecasting with scenarios, anomaly and driver detection, scheduled reports, alert rules, data-source connectors, billing tiers, and automated security scanning. The next frontier is the generative design vision (Section 14) and turning billing from a surface into real Stripe charges.

What NoPS is, in plain English

Imagine you run a business — a shop, a pharmacy, a small software company. You have a spreadsheet of every sale. It has the answers you need buried in it, but pulling them out means either learning to build charts yourself or paying an analyst.

NoPS is the analyst. You drag your spreadsheet onto the page, and within moments it:

- figures out what each column *means* (which one is revenue, which is the date, which is the customer, the region, the product...) — on its own;
- checks the data for problems (missing values, duplicates, typos, outliers) and offers to clean them — *without ever changing your original file*;
- builds a dashboard: the key numbers (revenue, profit, margin, orders, customers), how they changed vs. the previous period, trend lines, and rankings (top products, top customers, best regions);
- lets you ask questions in normal English and answers them;
- predicts where things are heading (forecasts);
- flags anything alarming automatically (a revenue drop, a looming stockout);
- and exports a clean PDF report you could hand to a board or a bank.

THE PROMISE ON THE TIN

“Upload business data → get automated dashboards, forecasts, alerts, a natural-language query interface, and board-ready PDF reports. Multi-tenant, role-based, works out of the box with sample retail data.” — that is the literal opening of the project’s README, and it’s accurate.

What makes it different from “just use Excel” or “just ask ChatGPT”

APPROACH	THE CATCH NOPS AVOIDS
Excel / Google Sheets	You still have to build every chart, formula and pivot yourself, and redo it each month.
Ask an AI chatbot to analyse the file	The AI can silently <i>invent</i> a number that looks plausible but is wrong — unacceptable for money or compliance. NoPS never lets AI produce a figure.
Hire a BI tool (Tableau, Power BI)	Powerful but heavy: you configure everything, it assumes you already know what you want. NoPS auto-generates the first useful view from the raw file.

So the identity is: **the speed and automation of an AI tool, with the trustworthiness of a spreadsheet formula you can audit line by line.**

The one big idea: “deterministic”

This word appears everywhere in the codebase. It is the product’s entire personality. Here’s what it means and why it was chosen.

The plain meaning

Deterministic = same input always produces the same output, forever, and you can read exactly how. The opposite is an AI language model, which can give slightly different (or wrong) answers to the same question and can’t show its arithmetic.

THE LINE THE WHOLE PRODUCT RESTS ON

“Every number is deterministic and reproducible. Schema detection, data profiling, forecasting, KPIs, and recommendations are all real code you can read, unit test, audit, and run offline — **no model produces a figure**. Given a question’s intent, the same dataset yields the same answer a million times.”

Why this is a genuinely smart bet

For the customer

- **Auditable** — every figure has a code path you can step through. Regulator- and finance-friendly.
- **Reproducible** — the same report next quarter is computed the same way.
- **Private** — your data rows never leave your servers.
- **Fast** — sub-100ms answers; no round-trip to a big AI model.

For the business (economics)

- **Free at rest** — the engine has no per-question AI bill, no rate limits, no vendor invoice.
- The *only* part that could ever cost per-use is the optional question-understanding AI, and it’s off unless a key is set.
- That means margins don’t erode as usage grows — the classic problem with AI-wrapper products.

So where *is* the AI, exactly?

In precisely one place: the natural-language chat box. When you type “top 10 customers,” the system needs to turn that English into a structured instruction. If an `OPENAI_API_KEY` is configured, it asks GPT to do that translation. If no key is set, a rule-based parser (regex and keywords) does it instead — fully offline. **Either way, the actual answer is computed by the deterministic engine, and the AI only ever sees your question text and your column names — never your data.**

THE MENTAL MODEL TO KEEP

AI is the *translator at the front desk*. The deterministic engine is the *accountant in the back room*. The translator never touches the ledger.

What it actually does — the full feature tour

Everything the app ships today, grouped by what you'd see as a user. Every item here is built and working.

AREA	WHAT YOU GET	STATUS
Accounts & teams	Sign up / log in / log out, forgot + reset password, secure tokens (JWT). Three roles — Admin (everything), Manager (uploads, analytics, reports, forecasts, chat), Viewer (read-only). Full “multi-tenant” isolation: every organisation’s data is walled off by an organisation ID.	BUILT
Data upload	Drag & drop CSV / XLSX / XLS. On arrival the file is parsed, profiled and quality-checked.	BUILT
Data-quality engine	Detects missing values, duplicates, empty columns, numbers-stored-as-text, statistical outliers, stray whitespace, inconsistent case/dates, suspicious column names. You accept or reject each cleaning suggestion — the original file is never modified.	BUILT
Schema detection	Automatically maps columns to business meaning: revenue, cost, profit, customer, product, region, date, inventory, and more.	BUILT
Auto dashboards	KPIs (revenue, profit, margin, orders, customers) with period-over-period comparison, revenue/profit trends, and rankings by product/customer/region/category. Generated from the detected schema — never hardcoded to one dataset.	BUILT
Analytics & filtering	Filter by date range, region, state, category, department, product, customer. Filters live in the URL so a view is shareable. Sortable/searchable data table + CSV export.	BUILT
Natural-language queries	Ask in plain English (“which month had the highest sales”, “which products are declining”). Mapped to a structured intent (by GPT if a key is set, else by rules), then executed through a controlled analytics layer. The model never runs code or sees your rows.	BUILT
Forecasting	Trend forecast with a 95% confidence band that widens with the horizon. Recently upgraded with seasonal model selection and best/base/worst scenarios .	BUILT
Anomaly & driver analysis	Deterministically flags unusual points and explains <i>what drove</i> a change (which product/region moved the number, with an “other” remainder so the parts reconcile).	BUILT
Correlation & segmentation	Finds relationships between metrics and splits customers into value tiers.	BUILT
Insights & alerts	Recommendations that carefully separate <i>observed data</i> → <i>possible cause</i> → <i>recommendation</i> . Automatic alerts for revenue drops, profit decline, inventory shortage, forecast risk, unusual performance.	BUILT

Scheduled reports & alert rules	A background scheduler regenerates reports on a cadence (hourly/daily/weekly/monthly) and optionally emails them; user-defined threshold rules raise alerts when a metric crosses a line.	BUILT
Executive reports	Structured report + server-side PDF export .	BUILT
Data-source connectors	Pull data from Postgres, MySQL, SQL Server or Google Sheets. Credentials stored encrypted (AES-GCM); an SSRF guard blocks connectors from reaching private hosts by default.	BUILT
Settings	Company profile, user management, API-key vault (stored hashed, never returned), dark/light theme.	BUILT
Billing tiers	Free / Pro (\$49) / Business (\$199) plans with enforced limits on datasets, seats and reports. Pricing page + usage limits are live; real Stripe charging is stubbed for wiring.	PARTIAL
Widget builder	A configurable “bento” dashboard (<code>/builder</code>): drag to reorder, resize, duplicate, remove, fullscreen, add from a palette; layout persists per user.	BUILT
Chart library	~14 chart types (combo, waterfall, funnel, scatter/bubble, radar, gauge, treemap, heatmap, donut, trend...) with a living catalog page at <code>/charts</code> .	BUILT

Who it's for & how it makes money

The intended customer

Small-to-mid businesses and teams that have data but no dedicated analyst, and — crucially — that **care whether the numbers are right and defensible**. The deterministic positioning specifically targets finance, regulated industries, and anyone who has to justify a figure to a board, an auditor, or a regulator. The seeded demo is a retail shop, but the app ships with **industry packs** that reshape the dashboard per business type.

Industry packs (how one app fits many businesses)

An “industry pack” is *data, not new code*. Each pack declares the extra vocabulary to recognise, which KPIs matter and how to compute them, and how to label the dashboard. Adding a new industry means adding one entry. Packs shipped today:

PACK	TUNED FOR	DISTINCTIVE KPIS
Retail / Sales (default)	Shops, e-commerce, general sales	Revenue, Profit, Orders, Customers, Margin
Pharmacy	Dispensaries, pharmacies	Prescriptions, Revenue, Patients...
SaaS	Subscription software	Subscription/recurring-revenue oriented
Generic	Anything unrecognised	Safe defaults from detected columns

The app can even auto-suggest the right pack by looking for distinctive “signal” columns in your data. Sample datasets ship for pharmacy and SaaS (plus a generated retail set) so you can see each pack in action.

The pricing model

PLAN	PRICE	DATASETS	SEATS	REPORTS / MONTH
Free	\$0	2	2	3
Pro	\$49 / mo	25	10	100
Business	\$199 / mo	Unlimited	Unlimited	Unlimited

HONEST STATUS ON BILLING

The *plans, limits and pricing page* are real and enforced — the app will stop a Free org at 2 datasets. The *actual credit-card charging* via Stripe is scaffolded (there are `stripeCustomerId` / `stripeSubscriptionId` fields and placeholders for price IDs) but not yet wired to live payments. This is the main gap between “usable app” and “can take money.”

The story so far — from commit #1 to today

84 commits over ~6 weeks tell a clear arc: build an MVP → harden it → make it a real product (money, marketing, teams) → deepen the analytics → design the ambitious future. Here are the milestones.

10 JUL 2026

Day zero

Initial commit. An empty canvas with a project description.

18 JUL 2026

The MVP lands — “DecisionIQ”

“Build DecisionIQ MVP: AI decision-intelligence platform.” The whole first version arrives at once, then is quickly made to serve its own web UI so the entire thing starts with one Docker command.

19 JUL 2026

The first hardening wave

Ten code-review fixes (security, permissions, correctness), then a three-part audit (backend, frontend, UX/accessibility).

Bugs fixed with dark-mode surfaces, stale reports, and an AI answer that was “confidently wrong.” The project’s own

`CLAUDE.md` guardrails are added.

20 JUL 2026

Doubling down on determinism

A pure-TypeScript **Statistical Engine** is added (zero dependencies). The product is explicitly repositioned as “a deterministic platform,” and unused LLM plumbing is dropped. Parsing, validation, race conditions and performance are all tightened.

25 JUL 2026

From tool to product

A premium dark-first redesign, then a burst of product-maturity work: **industry packs** (adapt per business), guided onboarding for new workspaces, **monetization** (plans, usage limits, billing surface), a public marketing landing page, and persistent chat history + a fixed invite flow.

26 JUL 2026

Connectors & polish

A collapsible full-screen sidebar; data-source connectors made functional (pull from real databases).

4 AUG 2026

Motion system

A framer-motion animation system across the web app — the “premium feel” layer.

18 AUG 2026

Optional AI + a test push + the component build-out

Optional AI question-understanding is added (translate-only, GPT). A big test-coverage effort adds unit + DB-integration tests and CI. Then a five-phase UI component build-out: DataTable, filter system, 8 new chart types, AI presentation components, and the widget/bento system. Finally, the product is **renamed from DecisionIQ to NoPS**, and the generative-design architecture doc is written.

18–19 AUG 2026

Security hardening

HTTP security headers, dependency-vulnerability patches, guarded file parsing, automated scanning (Dependabot), and a swap from the vulnerable `xlsx` to `@e965/xlsx`. Connectors hardened; tenant isolation proven with tests; dead code dropped.

19 AUG 2026

The analytics deepening (most recent work)

Deterministic **anomaly detection** and **driver analysis; correlation** and value-tier **segmentation**; forecasting upgraded with **seasonal models + scenarios**; all surfaced in the UI and via new API endpoints. Plus **scheduling**: scheduled reports and user-defined alert rules, with an automation UI and integration tests hardening the scheduler.

20 AUG 2026

Today

Latest merge (PR #41). The codebase is at its most capable — and this handbook is being written from it.

Current status — what’s real today

A candid map of what works, what’s half-done, and what's only designed. This is the “what the hell is actually happening” table.

Solid and shipping

CAPABILITY	STATE
Auth, roles, multi-tenant isolation (proven with tests)	BUILT
Upload → parse → profile → quality-check → schema-detect pipeline	BUILT
Auto dashboard with period-over-period KPIs, trends, rankings	BUILT
Deterministic analytics engine (KPIs, series, group-by, filters)	BUILT
Natural-language query (rules + optional GPT translation)	BUILT
Forecasting w/ seasonal selection + scenarios + confidence band	BUILT
Anomaly detection, driver analysis, correlation, segmentation	BUILT
Insights, automatic alerts, scheduled reports, alert rules	BUILT
PDF report export (server-side)	BUILT
Data-source connectors (Postgres/MySQL/SQL Server/Sheets), encrypted creds	BUILT
Industry packs (retail / pharmacy / saas / generic) + auto-suggest	BUILT
Component system: ~14 charts, DataTable, filters, widgets, AI cards	BUILT
Security headers, dependency scanning, guarded parsing	BUILT
CI running tests + typecheck on every push/PR	BUILT

Half-built / needs finishing

CAPABILITY	WHAT’S MISSING	STATE
Billing	Plans & limits enforced; live Stripe charging not wired	PARTIAL
Empty / error states	Primitives exist; per-context copy & typed variants to fill in	PARTIAL
Data prep UI	Quality panel exists; column mapper, transforms, formula builder to add	PARTIAL
Reports	Generate + PDF + scheduling done; report builder & templates to add	PARTIAL

Designed but not built

CAPABILITY	STATE
Generative dashboard design system (the big vision — Section 14)	PLANNED
Collaboration (share dialog, comments, mentions, activity feed, version history)	GAP
Advanced chart interactions (zoom, crosshair, drill-down, geo/choropleth)	GAP
Command palette / global search / saved-views selector	GAP

How it's built — the architecture

For the technically curious. Even if you're not, the shape here is simple and worth seeing once.

The project is a **monorepo** — one repository holding two applications that share tooling:

```
apps/
  api/           ← the "back end": Express + TypeScript REST API
    prisma/      schema, migrations, seed, sample-data generator
    src/
      engine/    THE DETERMINISTIC CORE — pure, testable functions
      auth/      JWT, requireAuth, requireRole
      modules/   one router per domain (uploads, datasets, analytics, ai, ...)
      ai/        optional GPT question-understanding (translate-only)
      billing/   plan definitions & limit checks
      scheduler.ts background jobs: scheduled reports + alert rules
  web/          ← the "front end": React + Vite + TypeScript + Tailwind
    src/{components,pages,lib}
docs/           architecture + roadmap docs (and this handbook)
docker-compose.yml one-command startup (Postgres + API + web)
```

The back end (apps/api)

A REST API in Node/Express/TypeScript. Its job: handle accounts and permissions, receive uploads, store data, and — above all — run the **engine**. Data lives in PostgreSQL, accessed through Prisma (a type-safe database toolkit). Uploaded rows are stored as JSON on the dataset record rather than in a per-cell table — a deliberate, simple choice that's correct up to ~100k-row files.

The front end (apps/web)

A single-page React app built with Vite. Tailwind for styling, Recharts for charts, TanStack Query for data-fetching, React Router for pages, framer-motion for the animation polish. Dark-mode-first. In production, the API even serves the built web files, so the whole product runs on one port.

The request flow, end to end

```
Upload a file
  → parse (CSV/XLSX → rows)
  → profile (stats + data-quality issues)
  → detect schema (which column means what)
  → store as a Dataset
Open the dashboard
  → engine computes KPIs, trends, rankings for the detected industry pack
  → forecast / insights / anomalies / drivers derived on demand
  → React renders it
Ask a question
  → (optional GPT) translate English → structured intent
  → validate intent against a closed vocabulary
  → deterministic analytics computes the answer
  → render as table / chart / metric
```

THE DESIGN PHILOSOPHY, STATED

“One deterministic implementation, no provider abstraction. Predictability *is* the product.” The team intentionally avoided speculative abstractions and kept the codebase lean.

The engine — the crown jewels

Everything valuable about NoPS lives in `apps/api/src/engine/`: ~2,400 lines of pure TypeScript with no I/O, no randomness, no external calls. Each file is one responsibility, and every bug ever fixed here got a permanent test (“regression assert”) so it can’t come back.

FILE	WHAT IT DOES
<code>parse.ts</code>	Turns a raw CSV/XLSX file into typed rows. Handles buffer validation, column inference, XLSX normalisation.
<code>profile.ts</code>	Profiles each column and finds data-quality issues (missing values, outliers, duplicates, type mismatches).
<code>schema.ts</code>	The “brain” that maps columns to business meaning (revenue, date, region, customer...). Also does row cleaning.
<code>analytics.ts</code>	The controlled query layer: KPI computation, time series, group-by aggregations, multi-value filters. Everything the dashboard and chat ultimately call.
<code>intent.ts</code>	Natural-language query → structured intent → answer. The closed-vocabulary safety net that keeps NL queries honest.
<code>forecast.ts</code>	Forecasting: seasonal model selection, trend, best/base/worst scenarios, and a residual-based 95% confidence band that widens with horizon.
<code>anomaly.ts</code>	Deterministic anomaly detection — flags unusual data points.
<code>drivers.ts</code>	Driver analysis — explains <i>what</i> moved a metric (which product/region), with an “other” remainder so the shown movers reconcile.
<code>correlate.ts</code>	Finds correlations between metrics.
<code>segment.ts</code>	Value-tier customer segmentation.
<code>insights.ts</code>	Turns all of the above into recommendations + alert seeds, carefully separating observation / cause / recommendation.
<code>industries.ts</code>	Industry packs — the single place a business type is defined (vocabulary, KPIs, labels).
<code>connectors.ts</code>	Pulls snapshots from external databases/sheets through a shared ingest pipeline.
<code>ingest.ts</code>	The shared path that takes any incoming rows and turns them into a profiled, schema-detected dataset.
<code>crypto.ts</code>	AES-GCM encryption for saved connector credentials.
<code>quantiles.ts</code>	Numerical primitives (quantiles, distribution math).
<code>report.ts</code>	Builds the structured executive report the PDF exporter renders.
<code>selfcheck.ts</code>	A runnable assert-based regression suite for the whole engine — the safety net.

Because these are *pure* functions (no database, no network, no clock, no randomness), they are trivially testable and provably repeatable — which is exactly what lets NoPS make its “every number is auditable” promise truthfully. Extending the product means adding one branch to one of these files plus one test. That’s a rare, healthy shape for a codebase this young.

The technology stack

What it's made of, and why each piece is there. All mainstream, well-supported, boring-in-a-good-way choices.

Front end (apps/web)

TECH	ROLE
React 18 + TypeScript	The UI framework and type safety.
Vite 6	Fast build tool / dev server.
Tailwind CSS 3	Styling via utility classes; design tokens for the dark-first theme.
Recharts 2	The charts (all ~14 types).
TanStack Query 5	Data fetching, caching, loading states.
React Router 7	Page navigation.
framer-motion 12	Animations (the “premium feel”).
lucide-react	Icon set.

Back end (apps/api)

TECH	ROLE
Node + Express 4 + TypeScript	The REST API server.
Zod	Validates every incoming request against a schema.
JWT + bcryptjs	Login tokens and password hashing.
Prisma 5 + PostgreSQL	Type-safe database access + the database itself.
multer + papaparse + @e965/xlsx	File uploads and CSV/XLSX parsing (the @e965 fork replaced xlsx to fix security advisories).
pdfkit	Server-side PDF report generation.
pg / mysql2 / mssql	Drivers for the data-source connectors.
nodemailer	Emailing scheduled reports.
openai	The optional question-understanding call (only if a key is set).
express-rate-limit	Throttling to prevent abuse.

Tooling & ops

TECH	ROLE
Docker Compose	One-command startup: Postgres + migrations + seed + API + web.
GitHub Actions (<code>ci.yml</code>)	Runs tests + typecheck on every push and PR.
Dependabot	Automated dependency-vulnerability alerts/updates.
Node's built-in test runner via <code>tsx</code>	Tests with no extra framework — lean by design.
Replit config	Cloud dev convenience.

The database — what’s stored

PostgreSQL, modelled with Prisma. Every table uses UUID IDs, has created/updated timestamps, is scoped to an organisation, is indexed, and cascades deletes correctly. Here’s the whole data model in plain terms.

TABLE	HOLDS
User	People: email, name, hashed password.
Organization	A workspace/tenant: name, industry pack, plan tier, plan status, Stripe IDs.
OrganizationMember	Links a user to an org with a role (Admin / Manager / Viewer).
PasswordResetToken	One-time tokens for the forgot-password flow.
Dataset	An uploaded/connected dataset: file info, row & column counts, quality score, detected columns, schema map, profiling stats, the original rows, and cleaned rows (JSON).
DataQualityIssue	Each detected problem: type, affected rows, severity, recommendation, whether auto-fixable.
Report	A generated report’s structured content.
Forecast	A saved forecast: metric, horizon, method, history, and points (with lower/upper bands).
Alert	A raised alert: type, severity, metric, value vs threshold, read/unread.
AIConversation / AIMessage	Chat history. (Historical names — they store user messages and the <i>deterministic engine</i> ’s structured responses. No AI-generated numbers are stored.)
ApiKey	The API-key vault: stored hashed, never returned; only the last four digits kept for display.
Connection	A saved data-source connector: type, non-secret config, AES-GCM-encrypted secret, last-sync status.
ActivityLog	Audit trail of actions per org.
ScheduledReport	A report the background scheduler regenerates on a cadence and optionally emails.
AlertRule	A user-defined threshold rule the scheduler evaluates (e.g. “alert if revenue < X, daily”).

Migrations are versioned in `apps/api/prisma/migrations/` — you can see the schema evolve: initial → connections → org industry → org plan → scheduling.

ONE DESIGN DECISION TO KNOW

Row data is stored as a JSON blob on the `Dataset` record, not spread across a per-cell table. That’s simpler and correct for MVP-scale files; the schema comments flag the move to a columnar warehouse if files ever exceed ~100k rows.

Security & privacy

For a young product this area is unusually mature — multiple dedicated hardening passes are in the history, and the sensitive paths are individually tested.

CONCERN	HOW IT'S HANDLED
Passwords	Hashed with bcrypt, never stored in plaintext.
Sessions	JWT tokens; the API refuses to boot in production with the default secret.
Tenant isolation	Every query is scoped by <code>organizationId</code> ; isolation is <i>proven with integration tests</i> , not just asserted.
Role enforcement	A <code>requireRole</code> gate on the routes; unit-tested.
Connector credentials	Encrypted at rest with AES-GCM; round-trip + tamper-detection tested.
SSRF (connectors reaching internal hosts)	A host guard blocks private/loopback targets by default (opt-in via env var for local demos); tested.
SQL safety	SQL-identifier quoting for connector table names; tested.
API keys	Stored hashed, never returned to the client; only last-four shown.
HTTP headers	Security response headers added to the API.
Upload safety	File parsing is guarded; max file size enforced (default 15 MB).
Dependencies	Dependabot scanning; the vulnerable <code>xlsx</code> package was swapped for the maintained <code>@e965/xlsx</code> fork.
Rate limiting	<code>express-rate-limit</code> throttles requests.
Data privacy vs AI	Only question text + column names are ever sent to OpenAI (and only if a key is set) — never data rows.

BOTTOM LINE

The security posture is a real asset. The main open item isn't a hole so much as an unfinished feature: production secrets management and the live-payments path both need the usual pre-launch attention.

Testing & quality

The testing culture here is a strong signal about the project’s health. It runs on every push and PR through GitHub Actions, with no heavyweight framework — just Node’s built-in test runner driven by `tsx`.

Two layers of tests

Engine regression suite

`selfcheck.ts` — assert-based checks covering profiling, schema detection, KPI period-over-period math, row cleaning, NL intent routing, forecasting, and insight derivation. The stated rule: **every bug fixed gets a new assert**, so regressions can’t silently return.

Unit & integration tests (`*.test.ts` / `*.itest.ts`)

Cover the security-critical paths *around* the pure engine: credential encryption, the SSRF guard, SQL quoting, the role gate, error-handler status mapping, the AI layer’s no-key privacy fallback, the billing/plan-limit money path, and DB-backed auth/RBAC/scheduler tests.

The 16 test suites in the repo

ENGINE	AROUND THE ENGINE
<code>analytics.test.ts</code> , <code>anomaly.test.ts</code> , <code>correlate.test.ts</code> , <code>drivers.test.ts</code> , <code>forecast.test.ts</code> , <code>parse.test.ts</code> , <code>segment.test.ts</code> , <code>crypto.test.ts</code> , <code>connectors.test.ts</code>	<code>auth/middleware.test.ts</code> , <code>billing/plans.test.ts</code> , <code>ai/provider.test.ts</code> , <code>errors.test.ts</code> , <code>scheduler.test.ts</code> , <code>app.itest.ts</code> , <code>schedules.itest.ts</code>

```
# run everything
npm test
# engine regression suite standalone
npx tsx apps/api/src/engine/selfcheck.ts
# on every push/PR: tests + typecheck via .github/workflows/ci.yml
```

WHAT THIS TELLS YOU

A project that tests its money path, its tenant isolation, and its privacy fallback — not just its happy path — is being built by someone thinking about production, not just a demo. That’s the single most reassuring thing in the repo.

The big future bet — a generative design system

There is a detailed 470-line architecture document (`docs/generative-design-architecture.md`) describing where the product wants to go next. It's the most ambitious idea in the project. Here's the whole thing in plain terms.

The problem it solves

Today, every business gets the *same-shaped* dashboard in the *same blue* — a fixed layout (KPI row → trend → donut → ranking) merely re-labelled per industry. A restaurant, a fintech and a logistics firm all get one skeleton. It's a strong baseline, but it's a template, not a design.

The vision

THE NORTH STAR

"NoPS is not a database of dashboards. It is a generative design system capable of composing **millions of valid business-intelligence experiences** from a constrained set of reusable primitives." An "AI Design Director" decides *what deserves attention and how to present it*; deterministic systems still decide *what the numbers actually are*.

How it would work (without breaking the trust promise)

The key insight is a strict split, enforced by a data contract:

- **The AI proposes presentation only.** It emits a `DashboardSpec` — a strict, versioned, JSON-schema-validated description of which components, layout, style and colours to use. It draws only from a *closed vocabulary* (it can't name a component or metric that doesn't exist).
- **The spec must never contain a number.** It carries only *references* (`metricId` , `seriesId` , `rankingId`). A deterministic renderer resolves those against the real engine outputs at render time. The AI literally cannot fabricate a figure.
- **Two gates.** A *Spec Validation Gate* (is it well-formed?) and a *Quality Engine* (is the resulting design actually good — hierarchy, readability, accessibility, density?). A design only ships if it clears both.
- **"Generate another design" is first-class.** A `DesignSeed` controls deterministic variation, so the same data can wear "minimal + spacious," "premium + editorial," "dense + executive," or "dark + futuristic" — same numbers, different identity.
- **A Color Intelligence Engine** builds accessible, industry-aware palettes (finance navy/gold, food warm terracotta...) — never "a random bright colour per card."

The rollout plan (phased, additive — not a rewrite)

A	The <code>DashboardSpec</code> + schema + component registry + a renderer that reproduces <i>today's</i> dashboard purely from a spec. Proves the boundary. (No AI yet.)
B	Color Intelligence + style families (runtime tokens, light/dark, accessibility gates).
C	The AI Design Director emits specs; add <code>DesignSeed</code> + “generate another design.”
D	The Quality Engine + scoring + repair/regenerate loop; v2 becomes default per-dataset with the pack dashboard as fallback.
E	(Future) a learning loop that ranks designs from real usage signals.

REALITY CHECK

This is **designed, not built**. It is the single most valuable and most risky piece of future work — valuable because “a dashboard that looks bespoke per business, with numbers you can still audit” is a genuinely differentiated product; risky because the Quality Engine (teaching software to judge “is this design good?”) is hard. The doc is refreshingly honest about the risks: regenerate-loop guards, quality-scoring performance, spec/renderer versioning.

Roadmap — what's next

Pulling together the two roadmap sources (the component roadmap and the generative-design phases) plus the obvious product gaps, here is a sensible ordering by leverage.

Near-term (finish what's started)

1. **Wire real Stripe billing.** The plans, limits and pricing page exist; connecting live checkout turns the app from “usable” to “revenue-capable.” Highest business leverage.
2. **Report builder + templates.** Generation, PDF and scheduling are done; a builder and templates make reports self-serve.
3. **Fill in empty/error-state polish and data-prep UI** (column mapper, transforms, formula builder) — the rough edges a paying user hits.

Medium-term (the differentiator)

1. **Generative design Phase A–B** — the `DashboardSpec` + renderer + color intelligence. Even without the AI Director, per-industry *visual* identity would be a visible leap.
2. **Chart interactions** — zoom, drill-down, crosshair, export, and geo/choropleth maps.
3. **Collaboration** — sharing, comments, activity feed, version history (currently the biggest untouched category).

Longer-term (the vision)

1. **Generative design Phase C–E** — the AI Design Director, the Quality Engine gate, and eventually a usage-driven learning loop.
2. **Scale the data layer** — move from JSON-on-row storage to a columnar warehouse when file sizes demand it.
3. **More industry packs** — each is just one data entry, so breadth is cheap.

THE ONE-LINE STRATEGIC READ

The foundation is done and unusually solid. The next dollar is best spent on **(a) turning on billing** and **(b) starting the generative-design layer** — the first unlocks revenue, the second unlocks a moat that competitors can't easily copy without also solving the “trustworthy numbers” problem NoPS already solved.

Every file, explained — useful vs. not

You asked to know “whatever the files are there, useful/unuseful.” The honest answer: this repo is **remarkably free of dead weight** — multiple commits explicitly removed audit docs and dead code. Almost everything here is load-bearing. Below is the guided tour.

Root — the control panel

FILE	PURPOSE	USEFUL?
README.md	The canonical, accurate product & setup doc.	Essential
package.json / package-lock.json	Monorepo scripts and locked dependencies.	Essential
docker-compose.yml	One-command startup (Postgres + API + web).	Essential
.env.example	Template for all configuration/secrets.	Essential
CLAUDE.md	Repo working agreement (code style, plan-before-coding).	Useful (process)
.replit	Replit cloud-dev convenience.	Optional
.github/	CI workflow + Dependabot config.	Essential (ops)

apps/api — the back end

PATH	PURPOSE
src/engine/*	The deterministic core (Section 9). The most valuable code in the repo.
src/modules/*	14 route files, one per domain: uploads, datasets, analytics, ai, forecasting, alerts, reports, schedules, connections, billing, organizations, settings, users, context.
src/auth/*	JWT middleware + auth routes (+ tests).
src/ai/provider.ts	The optional GPT question-understanding call (+ privacy-fallback test).
src/billing/plans.ts	Single source of truth for pricing/limits (+ test).
src/scheduler.ts	Background jobs for scheduled reports + alert rules (+ tests).
src/mailer.ts	Sends scheduled-report emails.
src/app.ts / index.ts / env.ts / errors.ts / prisma.ts	App wiring, entry point, env validation, error handling, DB client.
prisma/*	Schema, 5 migrations, seed script, sample-data generator.
src/sample/*	Sample datasets (pharmacy, SaaS CSVs) + generator so packs can be demoed.

apps/web — the front end

PATH	PURPOSE
<code>src/pages/*</code> (14)	Landing, Auth, Dashboard, DashboardBuilder, Analytics, DataList, DatasetDetail, AiChat, Forecasts, Alerts, Reports, ChartLibrary, Settings, Profile.
<code>src/components/*</code>	The UI system: charts, DataTable, filters, widgets, ai cards, Kpi, Shell, primitives, forms, navigation, feedback, schedules, analytics, onboarding, pricing, error boundary.
<code>src/lib/*</code>	Shared logic: api client, auth, theme, motion, kpi helpers, industries, types, utils.
<code>tailwind.config.js</code> / <code>index.css</code>	Design tokens + base styles.
Vite / TS / PostCSS configs	Build configuration.

docs/

FILE	PURPOSE	USEFUL?
<code>generative-design-architecture.md</code>	The big future vision (Section 14).	High (direction-setting)
<code>component-roadmap.md</code>	Honest inventory of built vs. remaining UI work.	High (planning)
<code>handbook.html</code> / <code>handbook.pdf</code>	This handbook.	You're reading it

THE “UNUSEFUL FILES” VERDICT

There essentially aren't any. The only *optional* items are `.replit` (cloud-dev convenience) and the sample CSVs (demo data you could delete in production). Everything else is doing a job. Past clutter (one-off audit/plan docs, a dead statistics library, dead UI controls, a CodeQL workflow that didn't apply) was *deliberately removed* along the way — which is itself a good sign.

How to run it yourself

Two ways. The first needs nothing but Docker and is genuinely one command.

Option A — Docker (easiest)

```
git clone https://github.com/kartikpolekar4518-hash/Business-Data-Analyst.git
cd Business-Data-Analyst
docker compose up --build
```

Wait for `NoPS API listening on :4000`, then open **http://localhost:4000**. Database, migrations, demo data, API and web UI all start automatically.

Option B — local development

```
docker compose up -d db          # Postgres only
cp .env.example apps/api/.env    # set at least JWT_SECRET
npm run setup                    # install + prisma generate + migrate + seed
npm run dev                      # api -> :4000, web -> :5173
```

Demo logins (workspace: “Acme Retail (Demo)”, password `password123`)

ROLE	EMAIL	CAN DO
Admin	<code>admin@nops.dev</code>	Everything: users, settings, API keys, uploads, reports, chat.
Manager	<code>manager@nops.dev</code>	Uploads, analytics, chat, reports, forecasts.
Viewer	<code>viewer@nops.dev</code>	View dashboards, reports, analytics.

The seed loads a realistic ~1,800-row retail dataset (18 months, 4 regions, 14 products) and writes it to a CSV so you can re-upload it through the UI.

Key configuration knobs (`.env`)

VARIABLE	WHAT IT CONTROLS
<code>DATABASE_URL</code>	Postgres connection string.
<code>JWT_SECRET</code>	Signs login tokens (required in production).
<code>OPENAI_API_KEY</code>	Optional. Set it to enable GPT question-understanding; leave blank for fully offline rule-based parsing.
<code>CONNECTOR_ENCRYPTION_KEY</code>	Encrypts saved connector credentials (required in production).

MAX_FILE_SIZE /

MAX_SYNC_ROWS

Upload and connector-sync size caps.

Honest assessment & risks

The part you actually need if you're deciding what to do next. Straight talk, grounded in what's in the repo.

Genuine strengths

- **A real product, not a demo.** Accounts, teams, permissions, billing tiers, connectors, scheduling — the boring-but-essential scaffolding of a SaaS is present.
- **A defensible, coherent thesis.** “Auditable numbers + optional AI convenience” is a real wedge into finance/regulated markets that pure AI tools can't claim.
- **Unusually clean engineering.** Pure engine, strong tests (including money path, tenant isolation, privacy fallback), CI, security hardening, and active removal of dead code.
- **Extensible by design.** New industry, insight, chart or intent = one small, tested addition.
- **A clear, ambitious, written-down future** that preserves the core promise.

Real risks & gaps

- **No live revenue path yet.** Billing is enforced but not charged. Nothing makes money until Stripe is wired.
- **Scale ceiling.** JSON-on-row storage is fine now, hard-capped later. Known and documented, but real.
- **The differentiator is unbuilt.** The generative-design vision is the moat, and it's the hardest, least-started work — especially the Quality Engine.
- **No collaboration features.** Sharing/comments/versioning is entirely absent — a common expectation for BI tools.
- **Single-author, AI-assisted history.** Fast progress, but bus-factor and long-term maintenance are worth planning for.

THE VERDICT IN ONE PARAGRAPH

NoPS is a well-engineered, feature-complete MVP with a sharp, defensible identity (trustworthy, deterministic analytics with optional AI convenience) and a genuinely ambitious roadmap (per-business generative design). The foundation is done and clean. The two things standing between it and being a real business are turning on payments and starting to build the visual-design differentiator. For a project barely six weeks old, that is an excellent place to be.

Glossary for non-technical readers

Every jargon word in this handbook, in plain English.

TERM	PLAIN MEANING
Deterministic	Same input always gives the same answer, and you can read exactly how. The opposite of an AI that can guess differently each time.
The engine	The trustworthy back-room code that does all the actual math.
MVP	Minimum Viable Product — the smallest version that’s genuinely useful.
Multi-tenant	One app safely serving many separate companies, each seeing only their own data.
KPI	Key Performance Indicator — a headline number like revenue or profit.
Schema detection	The app figuring out what each column in your spreadsheet means.
Profiling	Automatically summarising and quality-checking a dataset.
Forecast / confidence band	A prediction of future values, plus a shaded range showing how uncertain it is.
Anomaly	An unusual data point that stands out from the pattern.
Driver analysis	Explaining <i>what</i> caused a number to move (which product, region, etc.).
Industry pack	A small settings bundle that tailors the dashboard to a business type.
Connector	A live link to a database or Google Sheet so data flows in automatically.
API	The back-end service the app talks to; also how other software could plug in.
Front end / back end	Front end = what you see in the browser. Back end = the server doing the work.
Repository / monorepo	The folder holding all the code. “Mono” = both apps live in one repo.
Commit / PR	A commit is one saved change; a pull request (PR) is a reviewed batch of changes merged in.
CI	Continuous Integration — a robot that runs all the tests automatically on every change.
JWT / bcrypt / AES-GCM	Standard security tools: login tokens, password scrambling, and credential encryption.
SSRF	A type of attack where a server is tricked into calling internal systems; NoPS guards against it.
Stripe	The payments service that would handle credit-card charging.
DashboardSpec / Design Director	Future concepts: a strict recipe for a dashboard, and the AI that proposes it (without ever inventing numbers).